FIRST 10 PAGES OF SOURCE CODE • UNIFIED PROJECT DEPOSIT: SERVER.TS, INDEX.HTML, MAIN.TSX, SRC/APP.TSX

```
----  /* ======================================================================== */
----  /* FILE: server.ts (Lines: 431) */
----  /* ======================================================================== */
0001  import express from "express";
0002  import path from "path";
0003  import fs from "fs";
0004  import dotenv from "dotenv";
0005  import { createServer as createViteServer } from "vite";
0006  import { GoogleGenAI, Type } from "@google/genai";
0007
0008  dotenv.config();
0009
0010  const app = express();
0011  app.use(express.json());
0012
0013  const PORT = 3000;
0014
0015  // Local JSON Database Helper for Seeker Records
0016  const DB_DIR = path.join(process.cwd(), "data");
0017  const DB_FILE = path.join(DB_DIR, "seekers.json");
0018
0019  // Ensure data folder and seekers.json exist
0020  function initDatabase() {
0021    try {
0022      if (!fs.existsSync(DB_DIR)) {
0023        fs.mkdirSync(DB_DIR, { recursive: true });
0024      }
0025      if (!fs.existsSync(DB_FILE)) {
0026        fs.writeFileSync(DB_FILE, JSON.stringify([], null, 2), "utf-8");
0027      }
0028    } catch (error) {
0029      console.error("Failed to initialize seekers database:", error);
0030    }
0031  }
0032
0033  initDatabase();
0034
0035  // Load seekers from local JSON file
0036  function getSeekersList(): any[] {
0037    try {
0038      initDatabase();
0039      const data = fs.readFileSync(DB_FILE, "utf-8");
0040      return JSON.parse(data || "[]");
0041    } catch (error) {
0042      console.error("Error reading seekers database:", error);
0043      return [];
0044    }
0045  }
0046
0047  // Save seekers to local JSON file
```

FIRST 10 PAGES OF SOURCE CODE • UNIFIED PROJECT DEPOSIT: SERVER.TS, INDEX.HTML, MAIN.TSX, SRC/APP.TSX

```
0048 function saveSeekersList(seekers: any[]): boolean {
0049   try {
0050     initDatabase();
0051     fs.writeFileSync(DB_FILE, JSON.stringify(seekers, null, 2), "utf-8");
0052     return true;
0053   } catch (error) {
0054     console.error("Error writing to seekers database:", error);
0055     return false;
0056   }
0057 }
0058
0059 // Lazy initialize GoogleGenAI client to avoid crash if env is missing at startup
0060 let aiClient: GoogleGenAI | null = null;
0061
0062 function getGeminiClient(): GoogleGenAI {
0063   if (!aiClient) {
0064     const apiKey = process.env.GEMINI_API_KEY;
0065     if (!apiKey) {
0066       throw new Error("GEMINI_API_KEY environment variable is required. Please set it i...
0067     }
0068     aiClient = new GoogleGenAI({
0069       apiKey,
0070       httpOptions: {
0071         headers: {
0072           'User-Agent': 'aistudio-build',
0073         }
0074       }
0075     });
0076   }
0077   return aiClient;
0078 }
0079
0080 // Full-stack API Route for Tattoo Suggestion
0081 app.post("/api/suggest-tattoo", async (req, res) => {
0082   try {
0083     const {
0084       firstName,
0085       lastName,
0086       gender,
0087       birthDate,
0088       birthTime,
0089       birthPlace,
0090       intention,
0091       need,
0092       placementPreference,
0093       preferredLanguage = "English"
0094     } = req.body;
0095
0096     if (!birthDate || !intention || !need) {
0097       return res.status(400).json({ error: "Missing required fields for consultation." });
```

```
0098      }
0099
0100      const ai = getGeminiClient();
0101
0102      const userPrompt = `
0103        Please conduct a deep spiritual tattoo advisory consultation for a user with the ...
0104        - Full Name: ${firstName || "Unknown"} ${lastName || ""}
0105        - Gender Alignment: ${gender || "Unknown"}
0106        - Birth Date: ${birthDate}
0107        - Birth Time: ${birthTime || "Unknown"}
0108        - Birth Place: ${birthPlace || "Unknown"}
0109        - Primary Intention / Healing Goal: ${intention} (e.g. financial stability, emoti...
0110        - Emotional / Psychological Need: ${need} (e.g. healing trauma, self-love, overco...
0111        - Body Placement Preference: ${placementPreference || "Let the app choose"}
0112        - Preferred Language for explanation and mantras: ${preferredLanguage}
0113
0114        Based on this data:
0115        1. CRITICAL: All user-facing text properties inside the JSON (including 'summary'...
0116        2. Address the user directly by their first name (${firstName || "Seeker"}) throu...
0117        3. Recommended Tattoo Style: Choose the most spiritually and aesthetically aligne...
0118        4. Astrology Blueprint: Estimate the Sun, Moon, and Rising sign along with a domi...
0119        5. Life Area / Chakra Mapping: Identify the primary imbalanced or target Chakra, ...
0120        6. Custom Tattoo Concept: Detail a beautiful, symbolic tattoo design. Describe it...
0121        7. Dynamic Vector talismans in 3 Complexity Levels:
0122           Provide three alternative designs in the response:
0123           - Simple: Clean, minimalist, low line density, easy to tattoo (1-2 key sacred ...
0124           - Moderate: Balanced concentric design, medium line density (yantra/mandala/ge...
0125           - Advanced: Intricate, rich detailed layout, high line density (full cosmos co...
0126           Each of these MUST be valid SVGs, starting with "<svg viewBox='0 0 100 100' fi...
0127      `;
0128
0129      const response = await ai.models.generateContent({
0130        model: "gemini-3.5-flash",
0131        contents: userPrompt,
0132        config: {
0133          systemInstruction: `You are AstroPsyche, an enlightened spiritual tattoo adviso...
0134          You decode birth alignments, psychological wounds, and intention energy to desi...
0135          You must return the prescription in the exact JSON format requested. Your SVG m...
0136          CRITICAL RULE FOR DESIGN ELEMENTS: In the 'tattoo_concept.elements' list and al...
0137          responseMimeType: "application/json",
0138          responseSchema: {
0139            type: Type.OBJECT,
0140            properties: {
0141              astrology: {
0142                type: Type.OBJECT,
0143                properties: {
0144                  sunSign: { type: Type.STRING },
0145                  moonSign: { type: Type.STRING },
0146                  risingSign: { type: Type.STRING },
0147                  dominantPlanet: { type: Type.STRING },
```

```
0148                summary: { type: Type.STRING, description: "Detailed astrological align...
0149              },
0150              required: ["sunSign", "moonSign", "risingSign", "dominantPlanet", "summary"]
0151            },
0152            chakra_energy: {
0153              type: Type.OBJECT,
0154              properties: {
0155                targetChakra: { type: Type.STRING, description: "e.g., Root, Sacral, So...
0156                imbalanceAnalysis: { type: Type.STRING, description: "Detailed analysis...
0157                reikiSymbol: { type: Type.STRING, description: "Recommended Reiki symbo...
0158                vedicRemedy: { type: Type.STRING, description: "A protective Vedic mant...
0159              },
0160              required: ["targetChakra", "imbalanceAnalysis", "reikiSymbol", "vedicReme...
0161            },
0162            tattoo_concept: {
0163              type: Type.OBJECT,
0164              properties: {
0165                title: { type: Type.STRING },
0166                symbolism: { type: Type.STRING, description: "The deep symbolic narrati...
0167                style: { type: Type.STRING, description: "The recommended tattoo style,...
0168                elements: {
0169                  type: Type.ARRAY,
0170                  items: { type: Type.STRING },
0171                  description: "List of the concrete artistic elements included (e.g., ...
0172                },
0173                colors: { type: Type.STRING, description: "Ink color suggestions and th...
0174                placement: { type: Type.STRING, description: "The physical placement on...
0175                placementReason: { type: Type.STRING, description: "Energetic and physi...
0176                painIndex: { type: Type.INTEGER, description: "Estimated pain level fro...
0177                energySensitivity: { type: Type.INTEGER, description: "Energetic sensit...
0178                midjourneyPrompt: { type: Type.STRING, description: "A detailed design ...
0179                culturalRespect: { type: Type.STRING, description: "Mindful notes on th...
0180                svgMarkup: { type: Type.STRING, description: "A valid, beautiful, minim...
0181                svgMarkupSimple: { type: Type.STRING, description: "A simple version of...
0182                svgMarkupModerate: { type: Type.STRING, description: "A moderate comple...
0183                svgMarkupAdvanced: { type: Type.STRING, description: "An advanced, rich...
0184                simpleDesc: { type: Type.STRING, description: "A short 1-sentence poeti...
0185                moderateDesc: { type: Type.STRING, description: "A short 1-sentence poe...
0186                advancedDesc: { type: Type.STRING, description: "A short 1-sentence poe...
0187                size: { type: Type.STRING, description: "The recommended physical size ...
0188                direction: { type: Type.STRING, description: "The dynamic flow or rotat...
0189              },
0190              required: [
0191                "title", "symbolism", "style", "elements", "colors", "placement", "plac...
0192                "painIndex", "energySensitivity", "midjourneyPrompt", "culturalRespect"...
0193                "svgMarkupSimple", "svgMarkupModerate", "svgMarkupAdvanced", "simpleDes...
0194                "size", "direction"
0195              ]
0196            }
0197          },
```

```
          required: ["astrology", "chakra_energy", "tattoo_concept"]
        }
      }
    });

    const parsedData = JSON.parse(response.text || "{}");
    res.json(parsedData);
  } catch (error: any) {
    console.error("Error in tattoo suggestion API:", error);
    res.status(500).json({
      error: "We encountered a misalignment in the spiritual data streams.",
      details: error.message || error
    });
  }
});

// Full-stack API Route for Custom Sigil Analysis
app.post("/api/analyze-sigil", async (req, res) => {
  try {
    const { chakra, zodiac, planet, intention } = req.body;

    if (!chakra || !zodiac || !planet) {
      return res.status(400).json({ error: "Missing parameters for sigil analysis." });
    }

    const ai = getGeminiClient();

    const userPrompt = `
      Please channel a deep, mystical, and therapeutic energetic reading for a custom c...
      - Sacred Foundation: ${chakra}
      - Celestial Element: ${zodiac} Astrological Sign
      - Steering Frequency: ${planet} Planet
      - User's Personal Intention: ${intention || "General energetic centering & protec...

      Analyze how these three elements interact when layered together.
      Provide:
      1. A unique, mystical, Sanskrit-inspired or ancient-sounding Title for the combin...
      2. Resonance: How the sacred geometry of the chakra combines with the astrologica...
      3. Psyche Alignment: The psychological and somatic healing mechanism (how it rele...
      4. Practical Mantra: A short, elegant activation mantra that the user can chant d...
    `;

    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash",
      contents: userPrompt,
      config: {
        systemInstruction: `You are AstroPsyche, an enlightened spiritual tattoo adviso...
        You decode birth alignments, psychological blockages, and intention energies.
        Your tone is deeply poetic, comforting, sophisticated, and ancient, yet rooted ...
        Return the result in the exact JSON format requested. Do not include markdown w...
```

```
0248          responseMimeType: "application/json",
0249          responseSchema: {
0250            type: Type.OBJECT,
0251            properties: {
0252              title: { type: Type.STRING, description: "A mystical, elevated, or Sanskrit...
0253              resonance: { type: Type.STRING, description: "Poetic explanation of how the...
0254              psycheAlignment: { type: Type.STRING, description: "Detailed psychological ...
0255              practicalMantra: { type: Type.STRING, description: "A highly focused, beaut...
0256            },
0257            required: ["title", "resonance", "psycheAlignment", "practicalMantra"]
0258          }
0259        }
0260      });
0261
0262      const parsedData = JSON.parse(response.text || "{}");
0263      res.json(parsedData);
0264    } catch (error: any) {
0265      console.warn("Gemini sigil analysis failed, falling back to client-side explanation...
0266      res.status(500).json({
0267        error: "Celestial stream disconnected.",
0268        details: error.message || error
0269      });
0270    }
0271  });
0272
0273  // Database API: Save new seeker consultation record
0274  app.post("/api/save-seeker", (req, res) => {
0275    try {
0276      const {
0277        firstName,
0278        lastName,
0279        gender,
0280        birthDate,
0281        birthTime,
0282        birthPlace,
0283        intention,
0284        need,
0285        preferredLanguage,
0286        suggestion,
0287        deviceSignature
0288      } = req.body;
0289
0290      if (!birthDate || !intention || !suggestion) {
0291        return res.status(400).json({ error: "Missing essential seeker parameters to pers...
0292      }
0293
0294      const seekers = getSeekersList();
0295
0296      const newSeeker = {
0297        id: "seeker_" + Math.random().toString(36).substr(2, 9) + "_" + Date.now(),
```

```
      createdAt: new Date().toISOString(),
      firstName: firstName || "Anonymous",
      lastName: lastName || "",
      gender: gender || "Not Specified",
      birthDate,
      birthTime: birthTime || "Unknown",
      birthPlace: birthPlace || "Unknown",
      intention,
      need: need || "General spiritual enhancement",
      preferredLanguage: preferredLanguage || "English",
      deviceSignature: deviceSignature || "anonymous_soul",
      suggestion
    };

    seekers.unshift(newSeeker); // Add to the front of the array (newest first)
    const success = saveSeekersList(seekers);

    if (success) {
      res.json({ success: true, seeker: newSeeker });
    } else {
      res.status(500).json({ error: "Failed to write seeker record to file system." });
    }
  } catch (err: any) {
    console.error("Error saving seeker to database:", err);
    res.status(500).json({ error: "Internal server error during seeker registry.", deta...
  }
});

// Database API: Retrieve all seeker consultation records (supports filtering by device...
app.get("/api/get-seekers", (req, res) => {
  try {
    const { deviceSignature, passcode } = req.query;
    const seekers = getSeekersList();

    const masterPasscode = process.env.MASTER_LEDGER_PASSCODE || "AstroPsyche2026";
    const isMasterAuthorized = passcode === masterPasscode;

    // Filter seekers to secure individual records
    const securedSeekers = seekers.filter((s) => {
      // Authorized master practitioner can view everything
      if (isMasterAuthorized) return true;

      // If client signature matches, allow view
      if (deviceSignature && s.deviceSignature === deviceSignature) return true;

      // Backward compatibility: If no signature exists on older records, hide them fro...
      return false;
    });

    res.json(securedSeekers);
```

```
0348   } catch (err: any) {
0349     console.error("Error retrieving seekers from database:", err);
0350     res.status(500).json({ error: "Failed to retrieve seekers ledger.", details: err.me...
0351   }
0352 });
0353
0354 // Database API: Delete a seeker record (verifies signature or master passcode)
0355 app.delete("/api/delete-seeker/:id", (req, res) => {
0356   try {
0357     const { id } = req.params;
0358     const { deviceSignature, passcode } = req.query;
0359
0360     const masterPasscode = process.env.MASTER_LEDGER_PASSCODE || "AstroPsyche2026";
0361     const isMasterAuthorized = passcode === masterPasscode;
0362
0363     let seekers = getSeekersList();
0364     const targetSeeker = seekers.find((s) => s.id === id);
0365
0366     if (!targetSeeker) {
0367       return res.status(404).json({ error: "Seeker record not found." });
0368     }
0369
0370     // Security Gate: Check if current caller is owner of record or holds Master passcode
0371     const canDelete = isMasterAuthorized || (deviceSignature && targetSeeker.deviceSign...
0372     if (!canDelete) {
0373       return res.status(403).json({ error: "Access Denied: You are not authorized to re...
0374     }
0375
0376     seekers = seekers.filter((s) => s.id !== id);
0377     const success = saveSeekersList(seekers);
0378     if (success) {
0379       res.json({ success: true, message: "Seeker record released from cosmic ledger." });
0380     } else {
0381       res.status(500).json({ error: "Failed to update database file." });
0382     }
0383   } catch (err: any) {
0384     console.error("Error deleting seeker from database:", err);
0385     res.status(500).json({ error: "Internal server error during record deletion.", deta...
0386   }
0387 });
0388
0389 // Configure Vite middleware and static asset serving
0390 async function startServer() {
0391   if (process.env.NODE_ENV !== "production") {
0392     const vite = await createViteServer({
0393       server: { middlewareMode: true },
0394       appType: "spa",
0395     });
0396     app.use(vite.middlewares);
0397
```

```
0398      app.use("*", async (req, res, next) => {
0399        if (req.originalUrl.startsWith("/api")) {
0400          return next();
0401        }
0402        try {
0403          const indexPath = path.resolve(process.cwd(), "index.html");
0404          let template = fs.readFileSync(indexPath, "utf-8");
0405          template = await vite.transformIndexHtml(req.originalUrl, template);
0406          res.status(200).set({ "Content-Type": "text/html" }).send(template);
0407        } catch (e: any) {
0408          vite.ssrFixStacktrace(e);
0409          next(e);
0410        }
0411      });
0412    } else {
0413      const distPath = path.join(process.cwd(), "dist");
0414      app.use(express.static(distPath));
0415      app.get("*", (req, res) => {
0416        const indexPath = path.join(distPath, "index.html");
0417        if (fs.existsSync(indexPath)) {
0418          res.sendFile(indexPath);
0419        } else {
0420          res.status(404).send("Application dist build not found.");
0421        }
0422      });
0423    }
0424
0425    app.listen(PORT, "0.0.0.0", () => {
0426      console.log(`Server running on http://0.0.0.0:${PORT}`);
0427    });
0428  }
0429
0430  startServer();
0431
----  /* ========================================================================== */
----  /* FILE: index.html (Lines: 14) */
----  /* ========================================================================== */
0001  <!doctype html>
0002  <html lang="en">
0003    <head>
0004      <meta charset="UTF-8" />
0005      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
0006      <title>My Google AI Studio App</title>
0007    </head>
0008    <body>
0009      <div id="root"></div>
0010      <script type="module" src="/src/main.tsx"></script>
0011    </body>
0012  </html>
0013
```

```
0014
---- /* ========================================================================== */
---- /* FILE: src/main.tsx (Lines: 11) */
---- /* ========================================================================== */
0001 import {StrictMode} from 'react';
0002 import {createRoot} from 'react-dom/client';
0003 import App from './App.tsx';
0004 import './index.css';
0005
0006 createRoot(document.getElementById('root')!).render(
0007   <StrictMode>
0008     <App />
0009   </StrictMode>,
0010 );
0011
---- /* ========================================================================== */
---- /* FILE: src/index.css (Lines: 84) */
---- /* ========================================================================== */
0001 @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&fa...
0002 @import "tailwindcss";
0003
0004 @theme {
0005   --font-sans: "Inter", ui-sans-serif, system-ui, -apple-system, sans-serif;
0006   --font-serif: "Playfair Display", Georgia, serif;
0007   --font-mono: "JetBrains Mono", ui-monospace, monospace;
0008
0009   --color-mystic-950: #0b0c13;
0010   --color-mystic-900: #121422;
0011   --color-mystic-800: #1a1c30;
0012   --color-mystic-700: #252943;
0013   --color-gold-400: #e5c158;
0014   --color-gold-500: #d4af37;
0015   --color-gold-600: #aa8c2c;
0016   --color-violet-500: #8b5cf6;
0017   --color-emerald-500: #10b981;
0018 }
0019
0020 @layer base {
0021   body {
0022     background-color: var(--color-mystic-950);
0023     color: #f1f5f9;
0024     font-family: var(--font-sans);
0025   }
0026
0027   h1, h2, h3, h4, .serif-heading {
0028     font-family: var(--font-serif);
0029   }
0030 }
0031
0032 /* Custom micro-animations */
```